AutoSlim: Intelligent Automata Graph Optimization for Efficient Acceleration

TIFFANY YU and RASHA KARAKCHI (ADVISOR), University of South Carolina

Additional Key Words and Phrases: Automata Processing, Graph, FPGA, Artificial Intelligence, Random Forest, Acceleration

ACM Reference Format:
Tiffany Yu and Rasha Karakchi (Advisor). 2025. AutoSlim: Intelligent Automata Graph Optimization for Efficient Acceleration. In *Proceedings of The International Conference for High Performance Computing, Networking, Storage, and Analysis (SC'25)*.https://doi.org/XXXXXXX.XXXXXXX

1 Extended Abstract

Modern high-performance computing increasingly relies on sophisticated graph-based models to represent and manipulate symbolic data [1]. From bioinformatics and cyber security to inference of the AI model and text analytics, these applications often use directed graphs to capture complex dependencies and transitions between states [2–5]. However, as data sets and patterns grow in complexity and size, graph representations - composed of nodes and edges - also expand dramatically, resulting in excessive memory usage, power consumption, routing congestion, and inefficiencies in hardware acceleration platforms such as Field-Programmable Gate Arrays (FPGAs) [1, 6, 7].

These graphs often contain redundancies, such as rarely activated nodes, unnecessary connections, or expensive computation paths that contribute little to the final output, yet significantly bloat the hardware footprint.

To address this challenge, this paper presents **AutoSlim**, an intelligent automata graph optimization framework that leverages AI-based analysis to reduce the complexity of large-scale pattern matching graphs while maintaining their core functionality. At the heart of AutoSlim is a toolchain developed to generate a synthesizable benchmark suite of graph datasets with detailed annotations. These annotations go beyond standard state transition labels to include per-edge scoring metrics that indicate the frequency, likelihood, or computational cost of traversing a specific edge during matching operations [8, 9]. This scoring information is extracted from prior runs or static graph analysis and is instrumental in identifying nodes and edges with minimal contribution to system functionality.

The proposed framework then applies a supervised machine learning model - specifically a Random Forest classifier - trained to identify graph components with low impact [10–12]. The classifier distinguishes redundant or infrequently used edges and nodes based on various features such as transition frequency, edge weights, and connectivity patterns. Using this predictive capability, AutoSlim prunes the graph structure, removing edges and nodes that are unlikely to impact overall match accuracy or output correctness. Importantly, the framework includes verification steps to ensure that the pruned graphs preserve the functional integrity of the original design, particularly in preserving critical paths and reachable accepting states.

Authors' Contact Information: Tiffany Yu, tyu@email.sc.edu; Rasha Karakchi (Advisor), karakchi@cec.sc.edu, University of South Carolina.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
Manuscript submitted to ACM

The resulting optimized graphs are significantly more compact, with fewer nodes and interconnections, and are stored in a separate dataset repository to support extensive evaluation. The system conducts comprehensive comparisons between pre-pruned and post-pruned graphs by implementing both versions on a state-of-the-art FPGA-based automata processor known as NAPOLY+ [8, 9].

NAPOLY+ is a Nondeterministic Finite Automata (NFA) overlay architecture that utilizes distributed on-chip memory (BRAMs) to store state transition labels and allocates programmable logic (LUTs and registers) for interstate connectivity. Because hardware resources scale directly with the number of nodes and interconnections, the reductions achieved by AutoSlim translate into tangible improvements in FPGA utilization. Experiments conducted using a suite of biological and synthetic pattern datasets demonstrate up to 40% reduction in resource usage, alongside improvements in performance metrics such as power consumption, routing congestion, and throughput.

Moreover, AutoSlim helps reduce the time needed for compilation and reconfiguration phases, as simpler graphs mean fewer constraints on FPGA routing and placement. This accelerates the development lifecycle, particularly in edge computing environments, where frequent reconfiguration is needed. Beyond performance and efficiency, AutoSlim lays the foundation for more robust hardware security.

In summary, AutoSlim addresses a critical need in the high-performance computing landscape: how to intelligently reduce the size and complexity of graph-based accelerators without compromising functionality. By combining machine learning, graph analytics, and hardware-aware optimization, this work pushes the limit of what is achievable in symbolic data processing and pattern matching acceleration. The ongoing development includes integrating other ensemble AI models and benchmarking the effectiveness of AutoSlim in a variety of architectures beyond FPGAs, including CGRAs and memory-centric accelerators. One major future direction of this work is to enhance AutoSlim's AI model to not only prune redundant components but also to flag potential adversarial nodes or suspicious transitions. For instance, nodes representing symbols outside of an expected alphabet or transitions with atypical access patterns could indicate tampering or insertion of undesirable events (or intentionally or unintentionally malicious information). Incorporating this into the AutoSlim pruning pipeline would allow the tool to serve as a dual-purpose system: optimizing for performance and protecting against hardware-level threats.

2 Acknowledgments

This research was supported by McNair Junior Fellowship at Molinaroli College of Engineering and Computing at USC and supported for Magellan Scholar and Magellan Apprentice for Capstone Scholar by the USC Office of Undergraduate Research. The authors thank Rye Stahle-Smith and Darssan Eswaramoorthi for their assistance with hardware testing and experimental setup. The authors also thank AMD University Program for providing devices for this project.

References

[1] Rasha Karakchi and Jason D. Bakos. Napoly: A non-deterministic automata processor overlay. *ACM Transactions on Reconfigurable Technology and Systems*, 16:1–25, 2023.

[2] Henry Woods and Yannis Papakonstantinou. Automata processor applications in bioinformatics: A survey. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 15(5):1770–1782, 2018.

[3] Jordan A. Bradshaw, Rasha Karakchi, and Jason D. Bakos. Two-hit filter synthesis for genomic database search. In *2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, volume 52–55, pages 165–172, 2016.

[4] Rasha Karakchi, Jordan A. Bradshaw, and Jason D. Bakos. High-level synthesis of a genomic database search engine. In *2016 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*, pages 1–6, 2016.

[5] Rasha Karakchi, Charles Daniels, and Jason Bakos. An overlay architecture for pattern matching. In *2019 IEEE 30th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, volume 2160-052X, pages 165–172, 2019.

[6] Rasha Karakchi, Lothrop O. Richards, and Jason D. Bakos. A dynamically reconfigurable automata processor overlay. In *2017 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*, pages 1–8, 2017.

[7] Rasha Karakchi and Ryan Karbowniczak. Developing a self-explanatory transformer. In *2024 IEEE/ACM Symposium on Edge Computing (SEC)*, pages 523–525. IEEE, 2024.

[8] Ryan Karbowniczak and Rasha Karakchi. Optimizing sequence alignment with scored nfas. *arXiv preprint arXiv:2501.02162*, 2025.

[9] Ryan Karbowniczak Rasha Karakchi. A scored non-deterministic finite automata processor for sequence alignment. *arXiv preprint arXiv:2410.19758*, 2024.

[10] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[11] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

[12] Zhi-Hua Zhou and Ji Feng. Deep forest: Towards an alternative to deep neural networks. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 3553–3559, 2017.